

Hands-on 4: Difference Between JPA, Hibernate and Spring Data JPA

Modern Java applications use Object Relational Mapping (ORM) frameworks to simplify interaction with relational databases. JPA, Hibernate, and Spring Data JPA work together to reduce development effort.

Java Persistence API (JPA)

Java Persistence API (JPA) is a Java specification (JSR 338) that defines a standard approach for persisting, retrieving, and managing data between Java objects and relational databases.

JPA provides annotations such as `@Entity`, `@Table`, `@Id`, and `@Column` to map Java classes to database tables. Since JPA is only a specification, it does not perform database operations by itself and requires an implementation such as Hibernate.

Hibernate

Hibernate is a widely used Object Relational Mapping (ORM) framework and one of the implementations of the JPA specification. It converts Java objects into database records and automatically generates SQL queries for database operations.

Hibernate manages database connections, transactions, object mapping, caching, and query execution. Without Spring Data JPA, developers must explicitly create sessions, begin transactions, save entities, commit transactions, and close sessions.

Example (Hibernate):

```
Session session = factory.openSession();  
  
Transaction tx = session.beginTransaction();  
  
session.save(employee);  
  
tx.commit();  
  
session.close();
```

Spring Data JPA

Spring Data JPA is a Spring Framework module that simplifies database access by providing repository interfaces. It is built on top of JPA and works with JPA implementations such as Hibernate.

By extending `JpaRepository`, developers obtain built-in CRUD operations without writing SQL or Hibernate session management code. Spring also manages transactions through the `@Transactional` annotation.

Example (Spring Data JPA):

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
  
}  
  
@Service  
public class EmployeeService {
```

```
@Autowired

private EmployeeRepository employeeRepository;


@Transactional

public void addEmployee(Employee employee) {

    employeeRepository.save(employee);

}

}
```

Comparison

JPA	Hibernate	Spring Data JPA
Java specification	ORM framework	Spring module
Defines persistence standards	Implements JPA	Simplifies JPA usage
Does not provide implementation	Generates SQL and manages persistence	Provides repository interfaces and reduces boilerplate code
Requires an implementation	Works independently as an ORM	Uses JPA and Hibernate internally

Conclusion

JPA provides the standard specification for object-relational mapping, Hibernate implements the JPA specification to perform database operations, and Spring Data JPA further simplifies development by reducing boilerplate code through repository interfaces and automatic transaction management. Together, these technologies provide an efficient, maintainable, and scalable persistence solution for Spring Boot applications.